

Arbeiten mit der Nvidia DGX H100 „Deep-Thought“ an der DHBW Stuttgart

Last modified: 28 February 2025

Einleitung

Herzlich willkommen zur offiziellen Anleitung für die Nutzung der Nvidia DGX H100 „Deep-Thought“ an der DHBW Stuttgart. Dieses System steht den Studierenden für Machine-Learning-Projekte, datenintensive Berechnungen und andere wissenschaftliche Aufgaben zur Verfügung.

Die DGX H100 bietet Zugang zu leistungsstarken GPUs und ist über SLURM als Ressourcenmanager organisiert. Diese Anleitung erklärt, wie Sie:

- eine Remote-Entwicklungsumgebung mit Visual Studio Code (VSCode) einrichten,
- SLURM-Job-Skripte erstellen und konfigurieren,
- und SLURM-Jobs ausführen und überwachen.

Voraussetzungen

- Ein persönliches Benutzerkonto auf der DGX.
- Grundkenntnisse in Linux und Skripting (z. B. Bash, Python, Docker oder C).
- SSH-Zugang zur DGX H100.



Wichtige Information

Zum Ende jedes Semester findet ein CleanUp auf der DGX Statt um overhead zu vermeiden. Bitte denken Sie daran Ihre Ergebnisse immer direkt von der DGX Herunterzuladen. Vor durchführung eines CleanUp werden alle aktiven User informiert. Nach der Benachrichtigung haben Sie eine Woche Zeit ihre Projekte zu sichern.



Achtung

Es ist nicht gestattet auf der DGX direkt skripte ohne SLURM aus zu führen. Da sich das System aktuell in der Pilotphase befindet sind nicht alle Zugriffsrechte vollständig gesetzt. Nichtbeachten dieser Einschränkung führt zur Sperrung von Usern.

1. Einrichtung der VSCode Remote Development Umgebung

Visual Studio Code (VSCode) ermöglicht Ihnen, direkt auf der DGX H100 zu arbeiten, Code zu entwickeln und auszuführen.

Schritt-für-Schritt-Anleitung

1. VSCode installieren Laden Sie Visual Studio Code von <https://code.visualstudio.com/> herunter und installieren Sie es auf Ihrem lokalen Rechner.
2. Erweiterungen installieren Starten Sie VSCode und installieren Sie die folgenden Erweiterungen:
 - Remote - SSH: Für die Verbindung mit der DGX.
 - Docker: Zum Arbeiten mit Docker-Containern (falls nötig).
 - Python oder C/C++: Je nach Programmiersprache.
3. SSH-Zugriff einrichten

Öffnen Sie ein Terminal auf Ihrem lokalen Rechner und konfigurieren Sie den SSH-Zugriff:

```
ssh-keygen -t ed25519 -C "Ihr Benutzername"  
ssh-copy-id <Ihr-Benutzername>@141.31.112.54
```

Ersetzen Sie `<Ihr-Benutzername>` mit Ihrem LDAP-Benutzernamen.

3.5 ~/.ssh/config eintrag (Optional)

Im Terminal öffnen Sie die datei ~/.ssh/config

```
sudo nano ~/.ssh/config
```

Fügen Sie folgenden Eintrag hinzu:

```
Host deep-thought
```

```
HostName 141.31.112.54
```

```
IdentityFile ~/.ssh/<private-key-file>
```

```
User <username>
```

4. SSH-Verbindung in VSCode einrichten

- Öffnen Sie VSCode und wählen Sie Remote Explorer in der Sidebar.
- Fügen Sie ein neues Ziel hinzu:

```
ssh <Ihr-Benutzername>@141.31.112.54
```

Falls Sie mit Punkt 3.5 einen Eintrag in der ~/.ssh/config Datei hinzugefügt haben können Sie die Verbindung herstellen über:

```
ssh deep-thought
```

- Verbinden Sie sich mit der DGX. VSCode installiert automatisch die Serverkomponenten.

5. Arbeitsverzeichnis öffnen

Ihr persönliches Home-Verzeichnis wird automatisch erkannt. Sie können von dort aus Ihre Projekte starten.

2. Erstellung von SLURM-Jobfiles

SLURM-Jobfiles steuern die Ausführung Ihrer Skripte oder Programme auf der DGX. Sie sind einfache Shell-Skripte mit SLURM-spezifischen Direktiven.

Nomenklatur

Alle Slurm jobs benötigen einen Namen. Dieser muss für das Attribut `--job-name` gesetzt werden. Für alle Jobs gilt hier die folgende Nomenklatur:

`#USERNAME_#JOBNAME`

Beispiel 1: SLURM-Job für ein Python-Skript

```
#!/bin/bash
```

```
#SBATCH --job-name=#USERNAME_#JOBNAME    # Name des Jobs
#SBATCH --partition=gpu                    # Partition für GPUs
#SBATCH --gres=gpu:1                       # Anzahl der GPUs
#SBATCH --cpus-per-task=4                  # Anzahl der CPU-Kerne
#SBATCH --mem=16G                          # Arbeitsspeicher
#SBATCH --time=01:00:00                   # Maximale Laufzeit
#SBATCH --output=slurm-%j.out              # Ausgabeprotokoll (%j = Job ID)

# Python-Skript ausführen
module load python/3.10                    # Optional: Python-Modul laden
srun python3 ~/mein_projekt/script.py
```

Beispiel 2: SLURM-Job für ein C-Programm

```
#!/bin/bash
```

```
#SBATCH --job-name=#USERNAME_#JOBNAME
```

```
#SBATCH --partition=gpu
```

```
#SBATCH --gres=gpu:1
```

```
#SBATCH --cpus-per-task=2
```

```
#SBATCH --mem=8G
```

```
#SBATCH --time=00:30:00
```

```
#SBATCH --output=slurm-%j.out
```

```
# C-Programm kompilieren und ausführen
```

```
gcc ~/mein_projekt/program.c -o ~/mein_projekt/program
```

```
srunc ~/mein_projekt/program
```

Beispiel 3: SLURM-Job für docker

```
#!/bin/bash
```

```
#SBATCH --job-name=#USERNAME_#JOBNAME
```

```
# Name des Jobs
```

```
#SBATCH --partition=gpu
```

```
# Partition für GPU-Aufgabe
```

```
#SBATCH --gres=gpu:1
```

```
# Anzahl der benötigten GPU
```

```
#SBATCH --cpus-per-task=4
```

```
# Anzahl der CPU-Kerne
```

```
#SBATCH --mem=16G
```

```
# Arbeitsspeicher
```

```
#SBATCH --time=02:00:00
```

```
# Maximale Laufzeit
```

```
#SBATCH --output=slurm-%j.out.
```

```
# Ausgabeprotokoll (%j = Job ID)
```

```
# Define variables
```

```
IMAGE_NAME="my_docker_image"
```

```
CONTAINER_NAME="${IMAGE_NAME}_${date +%s}"
```

```
DOCKERFILE_PATH="." # Change if your Dockerfile is in a different
```

```
# Function to clean up the container on termination
```

```
cleanup() {
```

```
    echo "Caught termination signal (or program ended), stopping co
```

```
    docker stop "$CONTAINER_NAME"
```

```
    docker rm "$CONTAINER_NAME"
```

```
    exit 0
```

```
}

# Trap common kill signals
trap cleanup SIGTERM SIGINT SIGHUP EXIT

# Build the Docker image
echo "Building Docker image..."
docker build -t "$IMAGE_NAME:latest" "$DOCKERFILE_PATH"

# Check if build was successful
if [ $? -ne 0 ]; then
    echo "Docker build failed! Exiting..."
    exit 1
fi

# Run Docker in detached mode
echo "Running Docker container..."
docker run --rm \
    --gpus all \
    --name "$CONTAINER_NAME" \
    -v $HOME:/workspace \
    "$IMAGE_NAME:latest" \
    python3 /workspace/mein_script.py &
    # Use all gpus
    # Container name
    # Mount home directory
    # Image name

# Get container PID
CONTAINER_PID=$!

# Wait for the container to complete
wait $CONTAINER_PID
EXIT_CODE=$?

# Ensure cleanup happens if not trapped earlier
cleanup

exit $EXIT_CODE
```

Wichtig bei Slurm jobs die mit docker arbeiten ist die cleanup funktion zu

implementieren. Diese stellt sicher das container beendet werden sobald das programm fertig gelaufen ist oder abbricht. Zudem muss der Build prozess teil des Jobs sein.

3. Konfiguration von SLURM-Jobs

Wichtige SLURM-Parameter

Parameter	Beschreibung	Beispiel
--job-name	Name des Jobs (zur besseren Übersicht)	--job-name=test_job
--partition	Gewünschte Partition (z. B. gpu für GPU-Tasks)	--partition=gpu
--gres	Ressourcenzuweisung (z. B. Anzahl GPUs)	--gres=gpu:2
--cpus-per-task	Anzahl der benötigten CPU-Kerne	--cpus-per-task=4
--mem	Arbeitsspeicherbedarf	--mem=16G
--time	Maximale Laufzeit im Format HH:MM:SS	--time=01:00:00
--output	Speicherort der Ausgabe (Protokolle und Fehlermeldungen)	--output=job.out

Modulverwaltung

Verfügbare Module auf der DGX können mit dem folgenden Befehl aufgelistet werden:

```
module avail
```

Module können mit `module load <modulname>` geladen werden.

4. Ausführung und Überwachung von SLURM-Jobs

Job abschicken

Laden Sie Ihr SLURM-Jobfile hoch und führen Sie folgenden Befehl aus:

```
sbatch jobfile.slurm
```

Job-Status überprüfen

Verwenden Sie den folgenden Befehl, um den Status Ihrer Jobs zu sehen:

```
squeue -u <Ihr-Benutzername>
```

Job abbrechen

Falls Sie einen Job abbrechen müssen, verwenden Sie:

```
scancel <Job-ID>
```

5. Ausführung von SLURM-Jobs direkt aus VSCode

1. Jobfile erstellen Erstellen Sie ein neues SLURM-Jobfile in Ihrem Home-Verzeichnis, z. B. jobfile.slurm.
2. Terminal in VSCode verwenden Öffnen Sie ein Terminal in VSCode (`Ctrl+`) und navigieren Sie in Ihr Verzeichnis:

```
cd ~
```

3. Job starten Starten Sie Ihren Job mit:

```
sbatch jobfile.slurm
```

4. Ergebnisse analysieren Die Ausgabe wird standardmäßig in Ihrem Home-Verzeichnis gespeichert, z. B. als `slurm-<Job-ID>.out` .

Zusätzliche Hinweise

- Speicherbegrenzung: Beachten Sie die verfügbaren Ressourcen. Jobs, die zu viele Ressourcen anfordern, werden nicht ausgeführt.
- Dokumentation: Dokumentieren Sie Ihre SLURM-Jobs und deren Ergebnisse gut, um Fehler zu minimieren.
- Hilfe: Wenden Sie sich bei Problemen an den Ihre Laboringenieure oder Ihren

Betreuer.

Wir wünschen Ihnen viel Erfolg bei der Arbeit mit der DGX H100 „Deep-Thought“!